

Wind Data Quality Control

Overview

The Omnisense weather station (sensor `30B40014`) at the ARC ecovillage near Mkuranga, Tanzania uses a cup anemometer with a reed switch counter. Two categories of artefact appear in the raw data: **reed switch bounce** on the peak gust channel, and **isolated sensor spikes** on the average wind channel. This document explains how each is detected and corrected, and why the chosen approach is appropriate.

All three filters are applied uniformly across every project that consumes this sensor's data:

Project	Entry point
<code>arc_tz_weather</code>	<code>modules/common.py</code> → <code>wind_qc()</code> , called by <code>modules/wind.py</code> and <code>build.py</code>
<code>arc_tz_line</code>	inline in <code>load_weather_station_csv()</code> in <code>build.py</code>

Filter 1: avg_wind_kph isolated spike detection (rolling median ratio)

What it catches: Single-reading sensor errors on the average wind channel — for example, a jump to 70 km/h at 12:22 EAT on 13 April 2026 while all surrounding readings were near 5 km/h.

Why not a hard ceiling: A fixed ceiling (e.g. 60 km/h) would silently discard genuine extreme events. If a real storm builds the wind to 60+ km/h over several readings, that data is valuable and should be kept. A hard ceiling cannot distinguish “one bad reading” from “sustained high wind”.

The approach — rolling median ratio: For each reading, compute the median of the surrounding `AVG_SPIKE_WINDOW` (12) readings using a centred window (~1 hour at 5-minute intervals). Flag the reading if:

```
reading > AVG_SPIKE_RATIO * local_median    AND    reading > AVG_SPIKE_MIN_KPH
```

- **Why ratio 3x:** Gust spikes that are real events typically raise neighbouring readings too, so the local median rises with them and the ratio stays well below 3. An isolated single-point error (neighbours at 5 km/h, spike at 70 km/h) gives a ratio of ~14, caught easily.

- **Why 12-reading window:** A centred window of 12 means the spike itself is 1 of 12 values. The median is the 6th–7th sorted value, so a single outlier cannot shift it materially. The window spans ~1 hour, which is long enough to reflect ambient conditions without masking short real gusts.
- **Why 20 km/h minimum:** At very low wind speeds, small absolute values (e.g. avg = 1 km/h, median = 0.5 km/h) can produce large ratios from measurement noise. The minimum prevents false positives in near-calm conditions.
- **Genuine storm tolerance:** If wind genuinely climbs from 10 → 30 → 60 → 70 km/h over an hour, the local median tracks upward through each step. A reading of 70 km/h against a median of 50 km/h gives ratio 1.4 — well below the threshold. Only a sudden isolated jump is flagged.

Action: Flagged readings in `avg_wind_kph` are replaced with NaN / None and marked in `avg_wind_flagged`.

Filter 2: Reed switch bounce on `peak_wind_kph` (ratio + minimum)

What it catches: Mechanical bounce in the reed switch that counts anemometer cup revolutions. When the switch contacts momentarily chatter on a single closure, the firmware records a very high instantaneous peak speed even though the cup was barely moving. This produces rows where `peak_wind_kph` is many times larger than `avg_wind_kph`.

Two-part condition: `peak / avg > BOUNCE_RATIO (8) AND`
`peak > BOUNCE_MIN_PEAK_KPH (25 km/h)`

- The **ratio threshold of 8** distinguishes genuine gusty-wind events (typical gust factors of 1.5–3 in open terrain, rarely exceeding 5–6 even in convective storms) from bounce artefacts where the ratio reaches tens or hundreds.
- The **minimum peak of 25 km/h** prevents the filter from operating at near-zero speeds, where integer rounding in the firmware can create spuriously large ratios from tiny absolute values (e.g. avg = 0.1 km/h, peak = 1.0 km/h → ratio = 10, but both are essentially calm).
- Rows where `avg_wind_kph` is 0 or NaN (after spike filter above) are treated as an infinite ratio and flagged if `peak > 25 km/h`.

Why not “avg < 1.0 km/h AND peak > 25 km/h”: An earlier description of the bounce filter used a hard avg threshold. The ratio approach is strictly superior: it catches bounce at any average speed level (e.g. avg = 3 km/h, peak = 40 km/h → ratio 13), and the minimum-peak guard already handles the near-calm false-positive case.

Action: `peak_wind_kph` values satisfying the condition are replaced with NaN / None and marked in `peak_wind_flagged` . `avg_wind_kph` is not affected by this filter.

Filter 3: `peak_wind_kph` absolute ceiling (100 km/h)

What it catches: Any remaining implausibly high gust values that may not satisfy the ratio condition — for example, if both channels were simultaneously erroneous, or firmware producing very large integers.

Why 100 km/h: 100 km/h (~54 kn) is mid-Beaufort 10 (Storm). A genuine peak gust above this at this site would be an extraordinary event requiring independent verification. The ceiling is a safety net, not the primary filter.

Action: `peak_wind_kph` values above 100 km/h are replaced with NaN / None and marked in `peak_wind_flagged` .

Application order

Filters run in sequence so each step uses already-cleaned values:

1. **avg spike filter** (cleans `avg_wind_kph` first)
 2. **bounce ratio on peak** (uses the cleaned `avg_wind_kph` as denominator — important, since a flagged avg becomes NaN/None and is treated as infinite ratio)
 3. **peak ceiling** (independent, applied alongside step 2)
-

Constants

```
AVG_SPIKE_RATIO    = 3.0    # flag avg if > 3x local rolling median
AVG_SPIKE_WINDOW   = 12     # readings in rolling window (~1 hour at 5-min
                              intervals)
AVG_SPIKE_MIN_KPH  = 20     # minimum speed for ratio test to engage
PEAK_WIND_CEIL_KPH = 100    # km/h: hard ceiling for peak gust
BOUNCE_RATIO       = 8      # peak/avg ratio threshold for reed switch bounce
BOUNCE_MIN_PEAK_KPH = 25    # km/h: minimum peak speed for ratio filter to
                              engage
```

Any change to these values must be mirrored in `arc_tz_line/build.py` (inline in `load_weather_station_csv`).